



Universidad de Concepción

Tarea 1 - Forma O

Grupo 9

Benjamín Alonso Espinoza Henríquez - 2022451033

Nicolás López Cid - 2022423927

Sofía Ignacia López Aguilera - 2022451769

Profesor Gonzalo Pérez

Concepción, Chile

Septiembre 2026

Índice

1. Análisis del problema	3
1.1. Descripción del problema	3
1.2. Restricciones	3
1.3. Qué se decidió construir	3
2. Diseño de la solución	4
2.1. Diagrama de la arquitectura	4
2.2. Descomposición en servicios	4
2.3. Justificación de las fronteras	5
3. ADR	6
3.1. ADR-001 · Descomposición e integración (D1)	6
3.1.1. Contexto	6
3.1.2. Alternativas consideradas	6
3.1.3. Decisión	6
3.1.4. Justificación	6
3.1.5. Costo aceptado	6
3.1.6. Consecuencias	6
3.2. ADR-002 · REST frente a gRPC (D2)	7
3.2.1. Contexto	7
3.2.2. Alternativas consideradas	7
3.2.3. Decisión	7
3.2.4. Justificación	7
3.2.5. Costo aceptado	7
3.2.6. Consecuencias	8
3.3. ADR-003 · Contrato, versionado y evolución (D3)	8
3.3.1. Contexto	8
3.3.2. Alternativas consideradas	8
3.3.3. Decisión	8
3.3.4. Justificación	8
3.3.5. Costo aceptado	9
3.3.6. Consecuencias	9
3.4. ADR-004 · Resiliencia y modos de falla (D4)	9
3.4.1. Contexto	9
3.4.2. Alternativas consideradas	9
3.4.3. Decisión	9
3.4.4. Justificación	9
3.4.5. Costo aceptado	10
3.4.6. Consecuencias	10

4. Experimento	10
4.1. Hipótesis	10
4.2. Método	10
4.3. Datos	11
4.4. Conclusiones	11
5. Consideraciones	13
5.1. Factor elegido: privacidad de los datos de los usuarios	13
5.2. Cómo influyó en el diseño	13
5.3. Costos y límites	13
6. Instrucciones de ejecución	13
7. Reflexión final	14
7.1. Qué haríamos distinto	14
7.2. Qué quedó pendiente	14

1. Análisis del problema

1.1. Descripción del problema

La Red de Espacios de Trabajo Compartido CoLabora opera con dos sistemas que surgieron de forma independiente y nunca fueron integrados:

- **Espacios:** administra las salas y mantiene el conteo de puestos libres por franja horaria. Es la fuente de la verdad sobre la disponibilidad real.
- **Reservas:** registra a los usuarios y sus reservas de puesto, y que necesita conocer la disponibilidad de una sala antes de confirmar cada reserva

Al no existir comunicación automática entre ambos, la coordinación de puestos se resuelve hoy por chat. Esto introduce una condición de carrera humana: dos personas pueden reservar el mismo puesto sin que ningún sistema lo detecte, porque ninguno de los dos tiene visibilidad del estado del otro en tiempo real.

1.2. Restricciones

Además de los requisitos técnicos T1–T7 del encargo, el análisis mostró cuatro restricciones implícitas:

1. **Consultar y luego ocupar reproduce la doble reserva:** entre ambas llamadas, otro usuario puede tomar el último puesto.
2. **No hay transacción entre dos bases:** si la reserva no se guarda después de ocupar el puesto, queda un puesto ocupado sin reserva.
3. **Un timeout no significa que la operación no ocurrió:** Espacios pudo ocupar el puesto aunque la respuesta no llegara.
4. **Una dependencia caída no debe detener todo:** sin Espacios, iniciar sesión, ver las reservas propias y cancelar deben seguir funcionando.

Se asumió además que los usuarios se registran y reservan por sí mismos (anticipando el portal web que menciona el enunciado), que se reserva un puesto cualquiera de una sala en bloques de una hora (08:00–20:00) y que las salas son datos fijos de Espacios.

1.3. Qué se decidió construir

Dos servicios con base PostgreSQL propia: **Espacios**, servidor gRPC, y **Reservas**, API REST con autenticación JWT. Cada restricción implícita tiene una respuesta en el diseño:

1. Espacios verifica y ocupa en una sola transacción atómica.
2. Si algo falla después de ocupar, Reservas compensa liberando el puesto.
3. Cada ocupación lleva una referencia única, que hace idempotentes ocupar y liberar.

4. Reservas espera a Espacios un tiempo máximo, distingue sus fallas (503 y 504) y completa las liberaciones pendientes con una cola persistente.

Se implementaron también los opcionales O1 (caché con Redis), O2 (idempotencia), O3 (HA-TEOAS) y O4 (prueba contra el contrato), además de un frontend mínimo.

2. Diseño de la solución

2.1. Diagrama de la arquitectura

La Figura 1 muestra el sistema desplegado con Docker Compose. El patrón es REST hacia afuera, gRPC hacia adentro: los clientes externos solo hablan REST con Reservas, y solo Reservas habla gRPC con Espacios. Cada servicio tiene su propia base de datos, y ninguno accede a la del otro. El frontend se sirve con nginx, que además reenvía /v1 a Reservas, por lo que el navegador ve un único origen.

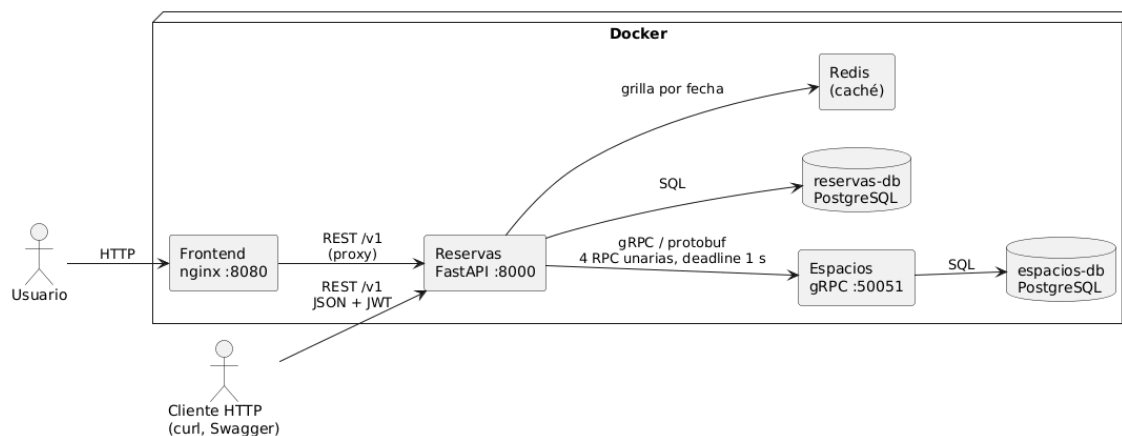


Figura 1: Arquitectura del sistema.

2.2. Descomposición en servicios

El sistema se divide en dos servicios, uno por cada sistema existente. La Figura 2 muestra cómo colaboran en el caso central que es reservar un puesto.

- **Espacios** responde qué hay, y cuánto queda libre. Administra salas, bloques horarios y ocupaciones, y se expone solo por gRPC (cuatro RPC unarias, uso interno). No sabe nada de Reservas porque de cada ocupación conoce solo una referencia opaca.
- **Reservas** responde quién reservó qué. Administra usuarios, reservas y la cola de liberaciones, y se expone como API REST pública bajo /v1, con autenticación JWT. De Espacios solo guarda el identificador de la sala y la franja de cada reserva.

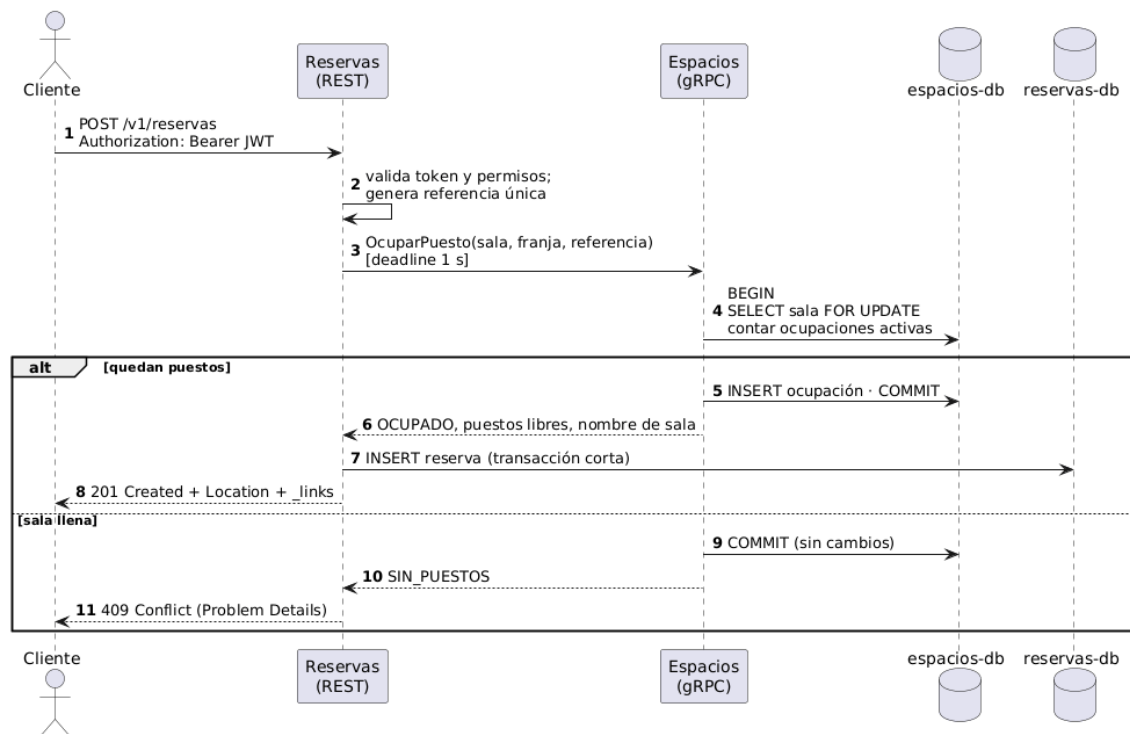


Figura 2: Reserva exitosa: Espacios verifica y ocupa en una sola transacción.

2.3. Justificación de las fronteras

Espacios habla de capacidad y ocupación, y Reservas de personas y compromisos. Tres consecuencias sostienen esta división:

- La decisión de si hay puesto la toma el dueño de la verdad. Espacios verifica y ocupa atómicamente, Reservas nunca decide con datos propios, ni siquiera con los de su caché.
- Espacios no recibe datos personales. Solo recibe "ocupa un puesto de la sala 3 el 1/10 de 09:00 a 10:00 con la referencia R".
- Los datos duplicados son mínimos y deliberados. Reservas guarda el identificador de la sala, la franja y una copia del nombre de la sala. Esto le permite listar las reservas aunque Espacios esté caído, a costa de que el nombre pueda quedar desactualizado.

3. ADR

3.1. ADR-001 · Descomposición e integración (D1)

3.1.1. Contexto

Espacios y Reservas son sistemas existentes, con equipos distintos. Hay que decidir si se mantienen separados y cómo se comunican, sin compartir base de datos.

3.1.2. Alternativas consideradas

- **Monolito:** una base y transacciones simples, pero obliga a reescribir dos sistemas y fusionar a sus equipos.
- **Monolito modular:** módulos separados en un solo despliegue, ambos equipos comparten ciclo de despliegue y no se escalan por separado.
- **Dos servicios con base propia:** respeta a cada dueño, a cambio de comunicarse por red y perder la transacción común. Considerando que la situación inicial era explícitamente sobre dos servicios que funcionaban sin comunicación, naturalmente surgió esta opción.

3.1.3. Decisión

Dos servicios con base propia, integrados por RPC síncrono (gRPC), con la frontera en el bounded context de cada uno. Espacios verifica y ocupa en una sola operación atómica mientras que Reservas solo conoce la sala y la franja de cada reserva.

3.1.4. Justificación

La decisión se tomó porque Espacios y Reservas tienen dueños y ciclos de cambio distintos. Una frontera alineada con esa organización permite que cada equipo modifique y despliegue su sistema sin coordinarse con el otro, salvo en el contrato.

Además, RPC síncrono se justifica en que una reserva requiere una respuesta inmediata sobre la disponibilidad, lo que excluye transferencia de archivos y mensajería asíncrona como opciones. La base compartida estaba denegada así que no se consideró. La invocación remota síncrona es el único estilo que cumplía con estas condiciones.

3.1.5. Costo aceptado

Todo lo que resuelve D4 es el precio de separar: latencia de red, fallas parciales, compensaciones en lugar de una transacción, y consistencia eventual. Además, seis contenedores y el nombre de la sala copiado en Reservas, que puede quedar desactualizado.

3.1.6. Consecuencias

El .proto pasa a ser la frontera. Con un solo equipo y un sistema nuevo, un monolito modular sería la opción más razonable y menos compleja.

3.2. ADR-002 · REST frente a gRPC (D2)

3.2.1. Contexto

El sistema tiene dos fronteras de comunicación con consumidores distintos. La externa (clientes hacia Reservas) la usan clientes que el equipo no controla, como el navegador y un futuro portal. La interna (Reservas hacia Espacios) tiene un único consumidor conocido, que la usa con mucha frecuencia y que el exterior no ve.

3.2.2. Alternativas consideradas

- **REST en ambas:** una sola tecnología, pero la comunicación interna pierde el contrato tipado y el control de tiempo por llamada.
- **gRPC en ambas:** una sola tecnología eficiente, pero el navegador no puede consumirla directamente y la API externa deja de ser legible con herramientas comunes.
- **REST hacia afuera y gRPC hacia adentro:** cada frontera con la tecnología que corresponde a su consumidor, a cambio de mantener dos tecnologías.

3.2.3. Decisión

REST con JSON para la API pública de Reservas y gRPC con protobuf para la comunicación interna entre Reservas y Espacios.

3.2.4. Justificación

La tecnología de cada frontera se elige según quién la consume. La frontera externa debe poder usarla cualquier cliente HTTP sin herramientas especiales ya que el navegador no habla gRPC sin un proxy intermedio, y REST además se depura y documenta con herramientas estándar. La frontera interna, en cambio, tiene un solo consumidor que genera su código desde el mismo contrato, por lo que puede aprovechar lo que gRPC ofrece, como tipos verificados en ambos lados, mensajes binarios compactos sobre conexiones HTTP/2 persistentes, y un tiempo máximo de espera deadline que viaja con cada llamada. Este último fue decisivo para D4, porque en el experimento, el deadline acotó el tiempo de respuesta de Reservas al valor configurado más unos 17 ms, sin importar cuán lento respondiera Espacios.

3.2.5. Costo aceptado

Dos tecnologías, dos contratos y dos conjuntos de herramientas. Reservas debe traducir los estados de gRPC a códigos HTTP. La comunicación interna es binaria, por lo que depurarla requiere herramientas específicas, y como la caché HTTP no aplica a gRPC, la caché se implementó en la aplicación (Redis).

3.2.6. Consecuencias

El navegador nunca se comunica con Espacios, por lo que Espacios podría cambiar de tecnología sin que los clientes externos lo noten, siempre que respete el `.proto`. Si Espacios llegara a tener consumidores externos, habría que exponerlo por REST o por gRPC-Web.

3.3. ADR-003 · Contrato, versionado y evolución (D3)

3.3.1. Contexto

El sistema tiene dos contratos con consumidores distintos, el `openapi.yaml` de la API pública, usado por clientes, y el `.proto` de Espacios, con un solo consumidor pero mantenido por otro equipo. Hay que definir qué cambios son compatibles, cuáles exigen una versión nueva y cómo se enteran los consumidores.

3.3.2. Alternativas consideradas

- **Versión en la ruta** (`/v1`), en un header o en el tipo de contenido. La ruta es visible y fácil de probar, las otras dejan URLs estables pero son invisibles al navegar.
- **Versión en el paquete del `.proto`** (`colabora.espacios.v1`). Con versión, dos versiones pueden servirse a la vez desde el mismo servidor.
- **Campos desconocidos**: ignorarlos o rechazarlos con 422, lo que detecta errores del cliente pero vuelve incompatible cualquier campo nuevo.

3.3.3. Decisión

Versión mayor en la ruta REST (`/v1`) y en el paquete gRPC, creando una versión nueva solo ante un cambio incompatible. Agregar campos, rutas o RPC es compatible; eliminar o renombrar campos, cambiar tipos o volver obligatorio un campo no lo es. En protobuf, los números de campo eliminados se reservan y nunca se reutilizan. Los campos desconocidos se ignoran, y los cambios se anuncian con marcas `deprecated`, un `CHANGELOG`, los headers `Deprecation` y `Sunset`, y un período en que ambas versiones conviven. Los contratos se escriben antes que el código, y una prueba automática verifica que la API implemente exactamente las rutas del contrato.

3.3.4. Justificación

Un cambio es compatible si un cliente anterior sigue funcionando sin modificarse. Agregar un campo solo cumple esa condición si ambos lados ignoran lo que no conocen, por eso el lector tolerante es requisito. En protobuf los campos viajan identificados por número y no por nombre, así que reutilizar un número haría que un cliente antiguo interprete un dato como otro sin ningún error. La versión en el paquete permite que Espacios publique una versión nueva sin obligar a Reservas a actualizarse el mismo día, lo que es necesario con equipos que no despliegan juntos.

3.3.5. Costo aceptado

La versión queda en la URL de cada recurso. Con el lector tolerante, un campo opcional mal escrito se descarta sin aviso. Mantener dos versiones vivas duplica código durante la transición, y escribir el contrato a mano exige una prueba que controle que no se desvíe de la implementación.

3.3.6. Consecuencias

No se construyó un /v2: la estrategia está documentada y se probó con los cambios compatibles del .proto. Todas las respuestas de error usan el formato estándar RFC 9457, que admite campos nuevos sin romper a los clientes.

3.4. ADR-004 · Resiliencia y modos de falla (D4)

3.4.1. Contexto

Reservas depende de Espacios de forma sincrónica. Espacios puede fallar de dos maneras: estar **caído**, y entonces la conexión no se establece, o estar **lento**, y entonces la respuesta no llega a tiempo. En el segundo caso el resultado es incierto: Espacios pudo haber ocupado el puesto aunque la respuesta nunca llegara.

3.4.2. Alternativas consideradas

- **Código de estado:** 500 (falso, no es un error de la API), 502 (Espacios no respondió mal, no respondió), siempre 503 (oculta la diferencia entre "no se hizo" y "no se sabe"), o 503 y 504 según el caso.
- **Cola de compensaciones:** en Redis o en un broker (sin transacción común con la base) o como tabla en la propia base de Reservas.

3.4.3. Decisión

Toda llamada a Espacios tiene un deadline de 1 s. Si Espacios no está disponible, o si el deadline vence sin haber establecido la conexión, se responde **503** ("no se hizo nada", con `Retry-After`); si vence con la conexión establecida, **504** ("resultado incierto"). Ante un 504 al reservar, se ordena liberar la referencia de esa ocupación, y Espacios registra la liberación para rechazar una ocupación tardía. Las liberaciones pendientes se guardan en una tabla de la base de Reservas (*transactional outbox*) que una tarea procesa cada 5 s, y cancelar una reserva siempre pasa por esa cola, por lo que funciona aunque Espacios esté caído. Dentro de Espacios, las esperas por bloqueos y por conexiones se acotan a 0,5 s, bajo el deadline, para responder "no se hizo nada" antes que Reservas se rinda.

3.4.4. Justificación

Distinguir 503 de 504 entrega al cliente la información que necesita para decidir si reintentar. El experimento midió el efecto del deadline: sin él, el tiempo de respuesta crece igual que la

lentitud de Espacios; con él, queda acotado al valor configurado más unos 17 ms. La regla de la conexión no establecida nació de una medición: al detener el contenedor de Espacios, resolver su nombre tardaba cerca de 4 s, más que el deadline, y el cliente recibía un 504 cuando en realidad no se había hecho nada; con la regla responde 503 en 1 s. Liberar por referencia es idempotente, así que la cola puede entregar la misma liberación más de una vez sin efectos extra, y al guardar la cancelación y la liberación en una misma transacción, ninguna liberación se pierde.

3.4.5. Costo aceptado

El timeout descarta respuestas que habrían llegado tarde, y un deadline cercano a la latencia habitual de Espacios corta casi todas las peticiones. Tras cancelar, el puesto se libera unos segundos después (consistencia eventual). Queda un riesgo residual: si después de ocupar fallan a la vez el guardado de la reserva, la liberación directa y el encolado, el puesto queda ocupado sin reserva.

3.4.6. Consecuencias

Con Espacios caído se puede iniciar sesión, listar y cancelar reservas; solo reservar y consultar disponibilidad responden 503.

4. Experimento

4.1. Hipótesis

Sin un tiempo máximo de espera, lo que tarda Reservas en responder crece igual que la lentitud de Espacios. Con un deadline D , la espera queda acotada cerca de D , a cambio de responder 504 en las peticiones que habrían terminado después de ese límite.

4.2. Método

Se midió GET /v1/salas (la grilla del día), con la caché desactivada para que cada petición llegara a Espacios, variando dos factores:

- **Latencia de Espacios:** 0, 250, 500, 750, 1000, 1500 y 2000 ms, agregada artificialmente al inicio de cada llamada gRPC.
- **Deadline de Reservas:** 500 ms, 1000 ms (el valor del sistema) y 30 000 ms, que representa "sin timeout".

Para cada una de las 21 combinaciones se reiniciaron Espacios y Reservas con esa configuración y se ejecutaron 3 corridas consecutivas de 200 peticiones, con 10 peticiones concurrentes, descartando las 10 primeras de cada corrida como calentamiento: 600 mediciones por combinación y 12 600 en total. Se registró el código de respuesta y el tiempo de cada petición, y se resumió cada combinación con la mediana (p50), el percentil 95 (p95) y el porcentaje de respuestas 200 y 504. Todo corrió en un mismo equipo (Windows 11, 12 hilos lógicos) con Docker Desktop, y el procedimiento completo es reproducible con `experimentos/timeout/medir.py`.

4.3. Datos

La Tabla 1 resume la mediana del tiempo de respuesta y el porcentaje de 504 en cada combinación. Las Figuras 3 y 4 muestran el mismo comportamiento de forma gráfica.

Latencia de Espacios	Deadline 500 ms	Deadline 1000 ms	Sin timeout
0 ms	45 ms · 0 %	83 ms · 0 %	54 ms · 0 %
250 ms	272 ms · 0 %	272 ms · 0 %	272 ms · 0 %
500 ms	517 ms · 99 %	521 ms · 0 %	520 ms · 0 %
750 ms	517 ms · 100 %	768 ms · 0 %	771 ms · 0 %
1000 ms	516 ms · 100 %	1018 ms · 97 %	1019 ms · 0 %
1500 ms	517 ms · 100 %	1018 ms · 100 %	1521 ms · 0 %
2000 ms	517 ms · 100 %	1018 ms · 100 %	2025 ms · 0 %

Cuadro 1: Mediana del tiempo de respuesta y porcentaje de respuestas 504 (600 peticiones por celda).

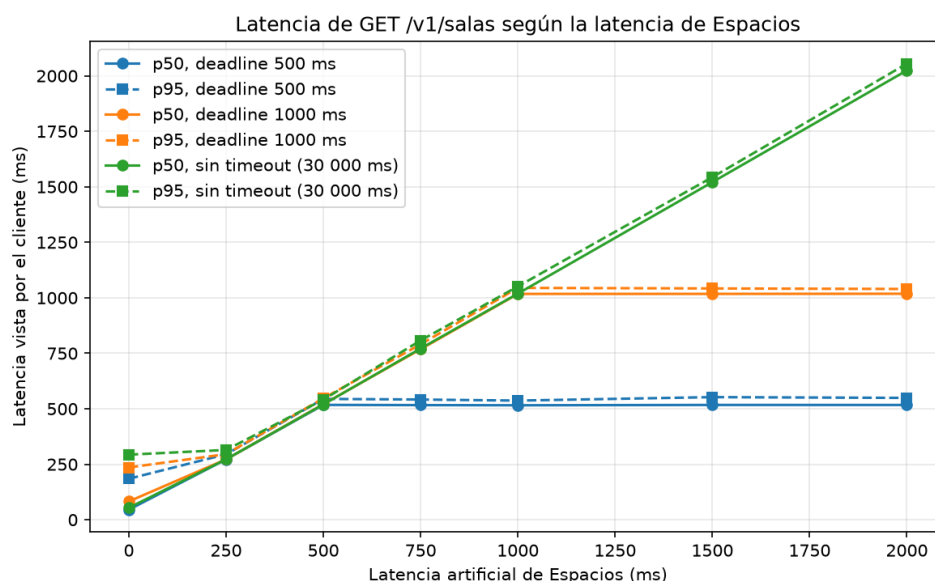


Figura 3: Tiempo de respuesta (p50 y p95) según la latencia de Espacios, para cada deadline.

4.4. Conclusiones

- **El deadline acota la espera.** Sin timeout, la mediana crece en línea recta con la latencia de Espacios (de 54 a 2025 ms). Con deadline, se detiene en el valor configurado más unos 17 ms (517 y 1018 ms), sin importar cuán lento responda Espacios. Se confirma la hipótesis.

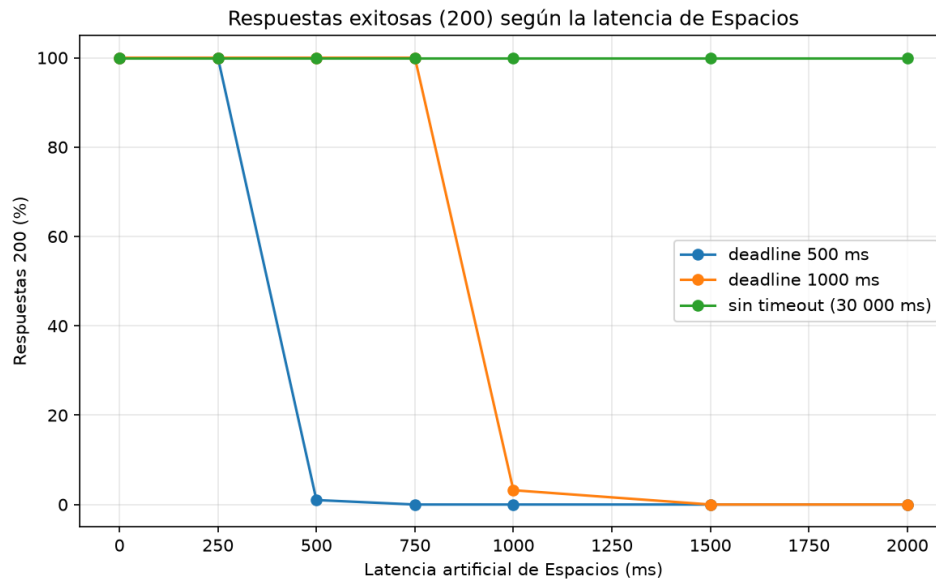


Figura 4: Porcentaje de respuestas exitosas según la latencia de Espacios, para cada deadline.

- **El costo es perder respuestas que habrían llegado.** Con deadline de 500 ms, una respuesta que tardaría 750 ms termina en 504, aunque Espacios la habría entregado.
- **El deadline debe fijarse con holgura.** Cuando la latencia de Espacios es igual al deadline (500/500 y 1000/1000), entre el 97 y el 99 % de las peticiones termina en 504, porque al tiempo de Espacios se suma el de la red y el procesamiento. Por eso el deadline de 1000 ms del sistema se fijó muy por encima de la latencia normal de Espacios, que está por debajo de 100 ms.
- **Las fallas se informan como se diseñó.** En todo el experimento no hubo ningún 503; Espacios nunca estuvo caído, solo lento, y cada falla se informó como 504, es decir, como un resultado incierto.

Limitaciones. Cliente y servicios corrieron en el mismo equipo, así que la red real está sobrerepresentada. La latencia artificial es constante, mientras que una latencia real varía entre peticiones. Las 3 corridas de cada combinación se hicieron seguidas sobre los mismos contenedores, no con reinicios independientes. Además, con 10 peticiones concurrentes y sin caché, el p95 con latencia 0 llegó a 185–293 ms frente a una mediana de 45–83 ms, lo que sugiere una cola de espera en el propio sistema. Por último, solo se midió la consulta de la grilla: no se midieron reservas, cuyos timeouts además generan compensaciones.

5. Consideraciones

5.1. Factor elegido: privacidad de los datos de los usuarios

Reservas guarda datos personales (nombre, email y contraseña) de usuarios que se registran por sí mismos. El factor elegido fue proteger esos datos, siguiendo un criterio general: cada componente debe conocer y exponer solo lo mínimo necesario para su función.

5.2. Cómo influyó en el diseño

- **Frontera entre servicios.** Los datos personales nunca salen de Reservas. Espacios solo recibe la sala, la franja y una referencia opaca (un UUID sin significado), por lo que una filtración de su base no expone a ninguna persona.
- **Contraseñas.** Se guardan solo como hash bcrypt y nunca se devuelven en una respuesta. Se limitaron a 72 bytes porque bcrypt ignora el resto, y así ninguna contraseña pierde en silencio parte de su valor.
- **El login no revela qué cuentas existen.** Una contraseña incorrecta y un email inexistente reciben la misma respuesta 401 y tardan lo mismo, porque en el segundo caso igual se calcula un hash de relleno. Así, midiendo el tiempo de respuesta no se puede averiguar qué emails están registrados.
- **El token no lleva datos personales.** El JWT es legible por cualquiera que lo obtenga, por lo que solo contiene el identificador y el rol del usuario, y expira a los 60 minutos.
- **Acceso solo a lo propio.** Un usuario que pide la cuenta o la reserva de otro recibe 404 y no 403, para no confirmar que ese recurso existe. El rol lo asigna siempre el servidor: enviar `rol": "administrador"` al registrarse no otorga privilegios.
- **Frontend.** Todo dato recibido de la API se muestra como texto y nunca como HTML, lo que impide que un contenido malicioso (por ejemplo, un nombre con código) se ejecute en el navegador de otro usuario.

5.3. Costos y límites

La privacidad entró en tensión con la trazabilidad: cada reserva registra a su titular y a quien la creó, lo que es necesario para auditar, pero conserva más datos personales. El sistema no permite eliminar cuentas, por lo que no atiende el derecho de supresión de datos que reconoce la Ley 21.719; la solución prevista es anonimizar la cuenta (reemplazar nombre y email) conservando sus reservas. Por último, en el entorno local la comunicación no está cifrada (HTTP sin TLS), por lo que en producción el uso de HTTPS sería obligatorio.

6. Instrucciones de ejecución

El sistema completo se levanta desde la raíz del repositorio. El único requisito es Docker con Docker Compose:

```
cp .env.example .env
docker compose up -d --build --wait
```

El comando construye las imágenes, crea las bases de datos con sus datos iniciales y espera a que todos los servicios estén listos. Quedan disponibles:

- Frontend: `http://localhost:8080`
- API REST y su documentación: `http://localhost:8000/docs`
- Espacios (gRPC, solo desde la máquina local): `127.0.0.1:50051`

El administrador inicial se crea con las credenciales `ADMIN_EMAIL_INICIAL` y `ADMIN_PASSWORD_INICIAL` del archivo `.env`. Con el sistema arriba, `python probar_sistema.py` ejecuta un recorrido completo, incluido el modo de falla (detiene y vuelve a iniciar Espacios). Para detener todo y borrar los datos: `docker compose down -v`. Las instrucciones para correr las pruebas y el experimento están en el `README.md` del repositorio.

7. Reflexión final

7.1. Qué haríamos distinto

- **Empezar más simple.** La primera versión tenía más capas, abstracciones y pruebas de las necesarias, y luego hubo que consolidar cerca de un tercio del código sin cambiar su comportamiento. Partir por lo mínimo y agregar estructura solo cuando hace falta habría ahorrado ese trabajo. También pensamos en que hubiera sido más fácil si planteábamos ambos sistemas aislados en primer lugar, aunque fueran "hardcodeados", para tener una idea del problema como tal, en vez de desarrollar la idea inmediatamente.
- **Probar el sistema completo antes.** Los hallazgos más importantes de resiliencia aparecieron al levantar todo con Docker, no en las pruebas unitarias, al detener el contenedor de Espacios hacía que la resolución de su nombre tardara unos 4 s y el cliente recibiera 504 en vez de 503, y el frontend respondía 502 si arrancaba antes que la API. Ambos se corrigieron, pero habrían aparecido antes con pruebas de sistema desde el inicio.
- **Reservar más tiempo para experimentar.** El experimento se diseñó desde el principio, pero dos de las tres mediciones planificadas (caché y tamaño de mensaje) se descartaron por tiempo. Con ellas, la caché (O1) tendría evidencia propia y D2 datos propios de tamaño.
- **Anticipar lo que el consumidor necesita del contrato.** Dos campos del `.proto` (iniciada y sala_nombre) se agregaron recién al implementar Reservas. Fueron cambios compatibles, pero diseñar el contrato junto con sus casos de uso los habría incluido desde el comienzo.

7.2. Qué quedó pendiente

- Reconciliación entre servicios, para eliminar el riesgo residual de un puesto ocupado sin reserva.

- Anonimización de cuentas, para atender el derecho de supresión de datos, y HTTPS en cualquier despliegue fuera del entorno local.
- Una prueba de contrato completa porque la actual verifica las rutas y los métodos, pero no que las respuestas cumplan los esquemas del `openapi.yaml`.
- Los experimentos descartados porque estaría bien conocer el efecto de la caché y el tamaño de los mensajes en protobuf frente a JSON.

Anexos

CoLabora

Sesión

Nombre: Administrador · Rol: administrador

Cerrar sesión

Disponibilidad

Fecha 12-02-2027 Ver grilla

Todas Sala 1 Sala 2 Sala 3 Sala 4

Sala	Franja	Puestos libres	
Sala 1	08:00–09:00	2 de 2	Reservar
Sala 1	09:00–10:00	2 de 2	Reservar
Sala 1	10:00–11:00	2 de 2	Reservar
Sala 1	11:00–12:00	2 de 2	Reservar
Sala 1	12:00–13:00	2 de 2	Reservar
Sala 1	13:00–14:00	2 de 2	Reservar
Sala 1	14:00–15:00	2 de 2	Reservar
Sala 1	15:00–16:00	2 de 2	Reservar
Sala 1	16:00–17:00	2 de 2	Reservar
Sala 1	17:00–18:00	2 de 2	Reservar
Sala 1	18:00–19:00	2 de 2	Reservar
Sala 1	19:00–20:00	2 de 2	Reservar

Todas las reservas

Actualizar					
Id	Sala	Fecha	Franja	Estado	
75	Sala 1	2026-10-25	08:00–09:00	CANCELADA	
74	Sala 1	2026-10-25	08:00–09:00	CANCELADA	
49	Sala 1	2027-02-06	09:00–10:00	ACTIVA	Cancelar
48	Sala 1	2027-02-06	09:00–10:00	ACTIVA	Cancelar
47	Sala 3	2027-02-06	08:00–09:00	ACTIVA	Cancelar

Usuarios

Actualizar			
Id	Nombre	Email	Rol
1	Administrador	admin@example.com	administrador
259	Demo	demo-ae8d2dca@example.com	usuario

Figura 5: Frontend preliminar del sistema.